From Reviews to Revisions: Closed-Loop Automation of Academic Paper Improvement

Gio Della-Libera
giodl@microsoft.com

April 24, 2026

Abstract

Traditional AI-assisted peer review systems generate structured feedback but leave the revision process to human authors, creating a gap between review and revision. We present a closed-loop automation system that bridges this gap by parsing review feedback, generating concrete revision plans, and applying edits directly to LaTeX source files. The system operates in three phases: (1) synthesis of reviewer feedback into priority-classified revision items (P1/P2/P3), (2) generation of REVISION-PLAN.md with specific edit locations and instructions, and (3) automated application of edits using structured code transformation tools. Evaluation on 14 research papers (186 reviews, 33 revision cycles) shows the system successfully completes 78% of P1 items automatically, reducing author revision time by 64% while maintaining edit quality (94% author acceptance rate). The system demonstrates that the gap between review generation and code modification is tractable for structured documents like LaTeX papers, with implications for automated code review and documentation maintenance. Key contributions include the first end-to-end closed-loop system for paper revision, empirical analysis of revision completion patterns, and a reusable architecture for structured document transformation.

1 Introduction

AI-assisted peer review has made substantial progress in generating structured, expert-level feedback for research papers [**?**, **?**]. These systems successfully identify weaknesses, suggest improvements, and classify issues by priority. However, they share a critical limitation: **the revision process remains entirely manual**. After generating reviews, authors must interpret feedback, locate the relevant LaTeX code, and apply edits themselves. This creates a bottleneck: the system can produce 20+ pages of detailed feedback in minutes, but authors spend days or weeks implementing revisions.

This paper addresses the question: **Where is the boundary between automatable and human-essential revision tasks?** When can an AI system safely apply edits directly to LaTeX source, and when must authors retain control? Understanding this boundary has implications for human-centered AI design: how do we build systems that augment authorial agency rather than replace it?

We present a **closed-loop automation system** that bridges the review-revision gap through three phases:

1. **Synthesis**: Consolidate reviewer feedback into SYNTHESIS.md with priority classification (P1: critical, P2: important, P3: nice-to-have) using methods from prior work [**?**].

2. **Planning**: Generate REVISION-PLAN.md with concrete edit instructions including file paths, line numbers, old text, and new text for each revision item.

3. **Execution**: Parse REVISION-PLAN.md and apply edits to LaTeX files using structured code transformation tools (Claude Code's Edit tool, enabling precise string replacement with diff validation).

The system operates iteratively: after applying edits, papers are re-reviewed, new feedback is synthesized, and the cycle repeats until convergence (average score $\geq$ 2.5/4, no score ¡ 2/4).

1.1 Key Challenges

Closing the loop requires solving three technical challenges:

(1) Unambiguous edit specification: Review feedback is often abstract ("improve clarity of methodology section"). The system must translate this into concrete edits ("replace lines 127-132 in sections/03-methodology.tex with the following text...").

(2) Precise code localization: The system must identify the exact file, line range, and text to modify. LaTeX papers have complex structure (main.tex includes sections/*, figures in separate files, macros, etc.), making naive pattern matching brittle.

(3) Edit validation and rollback: Automated edits can introduce LaTeX syntax errors or semantic inconsistencies. The system must validate edits (checking LaTeX compilation) and support rollback if errors occur.

1.2 Contributions and Findings

We evaluate the system on 14 papers across 3 research modules (merit, waves, basecamp) over 4 months, comprising 186 reviews and 33 revision cycles. Key findings:

- **78% P1 completion rate**: The system automatically completes 78% of P1 (critical) revision items without human intervention. The remaining 22% require human judgment (e.g., "restructure paper flow", "expand related work with 3+ new references").
- **64% time reduction**: Authors report 64% reduction in revision time (from 8.2 hours/paper avg to 2.9 hours/paper avg), as the system handles mechanical edits (fix typos, improve phrasing, add citations) while authors focus on substantive changes (add experiments, restructure arguments).
- **94% acceptance rate**: Authors accept 94% of automated edits without modification, indicating high edit quality. Rejected edits (6%) primarily involve subjective phrasing choices or domain-specific terminology.
- **Edit type distribution**: 42% of automated edits are phrase replacements ("replace X with Y"), 31% are insertions ("add text at line N"), 18% are deletions, and 9% are block reorderings (moving paragraphs or sections).
- **LaTeX compilation success**: 91% of automated revision cycles compile successfully on first attempt. Failures (9%) are primarily due to unmatched braces or figure path errors, detected immediately and rolled back.

This work makes three contributions:

1. **Mapping the automation boundary**: Empirical analysis of which revision tasks are safely automatable (78% of critical items: mechanical corrections, evidence insertion) vs. which require human judgment (22%: conceptual refinement, structural changes). This taxonomy informs human-centered design of writing assistance tools.
2. **The first end-to-end closed-loop system** bridging AI review generation and revision implementation, demonstrating that the gap is tractable for structured documents while revealing where automation threatens authorial agency.

3. **Design principles for human-centered automated revision**: Transparency (show why edits are suggested), selective automation (respect author preferences), and edit provenance (maintain traceable history). These principles extend to any domain where AI systems modify human-authored artifacts.

The remainder of this paper is organized as follows: Section 2 surveys related work in automated program repair and document transformation. Section 3 details the three-phase architecture (synthesis, planning, execution) and edit validation protocols. Section 4 presents evaluation across 14 papers. Section 5 analyzes trade-offs and design decisions. Section 6 concludes with future work and generalization to other domains.

2 Related Work

2.1 AI-Assisted Review Generation

Recent work demonstrates AI's capability to generate structured paper reviews matching expert quality [**?**]. Our prior work [**?**] presents an 8-stage lifecycle for AI-simulated reviews, achieving 32% score improvement across 14 papers. However, these systems generate feedback without implementing revisions, leaving a manual gap between review and revision.

Yuan et al. [**?**] use LLMs to generate meta-reviews consolidating multiple reviews. While this aggregates feedback, it does not translate recommendations into code edits. Our system extends this by generating executable revision plans with specific file paths and text replacements.

2.2 Automated Program Repair

Automated program repair (APR) systems fix bugs by generating and validating code patches [**?**]. GenProg [**?**] uses genetic programming to evolve patches that pass test suites. More recent neural approaches [**?**] use sequence-to-sequence models to generate fixes.

Our system applies APR techniques to LaTeX documents rather than source code. The key difference: LaTeX compilation provides weaker validation than test suites (it checks syntax but not semantics), requiring additional quality assurance (reviewer re-assessment after edits).

2.3 Code Refactoring and Transformation

Refactoring tools like IntelliJ IDEA and Eclipse automate code transformations (rename variable, extract method) using abstract syntax tree (AST) manipulation [**?**]. These tools guarantee behavior preservation through static analysis.

LaTeX lacks a standard AST representation, making AST-based refactoring infeasible. Our system uses structured string replacement with diff validation: match exact text spans, apply replacements, verify compilation. This is less rigorous than AST manipulation but sufficient for LaTeX's relatively simple syntax.

2.4 Document Editing and Revision Tracking

Track changes in Microsoft Word and Google Docs enable collaborative document editing with revision history [**?**]. These systems track who made which edits and when, supporting rollback and merge conflict resolution.

Our system generates edits automatically rather than tracking human edits. However, we adopt the revision tracking model: each edit is logged with justification (which review item prompted it), enabling rollback if edits are rejected.

2.5 Crowd-Based Revision and Professional Editing

An alternative to automated revision is human-powered editing through crowdsourcing or professional services. Platforms like Upwork, Fiverr, and Scribbr connect authors with editors who provide proofreading, copyediting, and substantive editing services [**?**]. Turnaround times range from 24 hours (expedited) to 1 week (standard), with costs from $50 (basic proofreading) to $1000+ (substantive editing).

Soylent [**?**] demonstrated crowd-powered word processing where mechanical tasks (shorten text, proofread) are outsourced to Amazon Mechanical Turk workers. This hybrid model achieves quality comparable to expert editors at lower cost, but with higher latency (minutes to hours per task).

Comparison to automated revision: Crowd-based editing offers human judgment and domain expertise that automation lacks, particularly for subjective tasks (improve flow, strengthen argument). However, crowds cannot match automation's speed (minutes vs. days) or cost ($0.50 API cost vs. $500 editor fee). The optimal workflow may be *hybrid*: automate mechanical edits (typos, citations), outsource complex edits to crowds or professionals, and reserve author effort for conceptual refinement.

Collaborative platforms like Overleaf enable co-authors or advisors to directly edit drafts [**?**], providing instant feedback at zero marginal cost. This is the fastest and cheapest alternative, but requires availability of knowledgeable collaborators willing to spend time on revisions.

Our system targets a different point in the design space: fully automated revision for mechanical tasks, with 64% time reduction at minimal cost ($12 per paper in API fees, see Section 5). We trade human judgment (crowds/professionals) for speed and cost efficiency, accepting that 22% of tasks still require author judgment.

Recent work shows LLMs can edit code when prompted with specific instructions [**?**]. Copilot [**?**] generates code completions, while Cursor and Aider [**?**] enable conversational code editing.

Our system differs in two ways: (1) edits are driven by structured review feedback rather than freeform prompts, and (2) we use deterministic string replacement (Edit tool) rather than generative completion, ensuring edits exactly match the revision plan.

2.6 LaTeX-Specific Tools

Overleaf [**?**] provides real-time collaborative LaTeX editing with change tracking. Grammarly and LanguageTool [**?**] detect grammar and style issues in text, including LaTeX.

These tools focus on syntax/grammar checking or human collaboration, not automated revision implementation from structured feedback. Our system integrates review synthesis (identifying what to change) with edit execution (making the change), closing the loop from feedback to code.

2.7 Gaps in Existing Work

No prior work combines AI review generation, concrete revision planning, and automated edit execution for academic papers. Existing systems either: (1) generate feedback without implementing it (AI review), (2) implement fixes without high-level feedback (APR), or (3) support human editing without automation (collaborative editing tools). Our contribution is an end-to-end closed-loop system spanning all three phases.

3 Methodology

3.1 System Architecture Overview

Figure 1 illustrates the three-phase closed-loop architecture. Each review cycle proceeds through synthesis $\rightarrow$ planning $\rightarrow$ execution $\rightarrow$ validation.

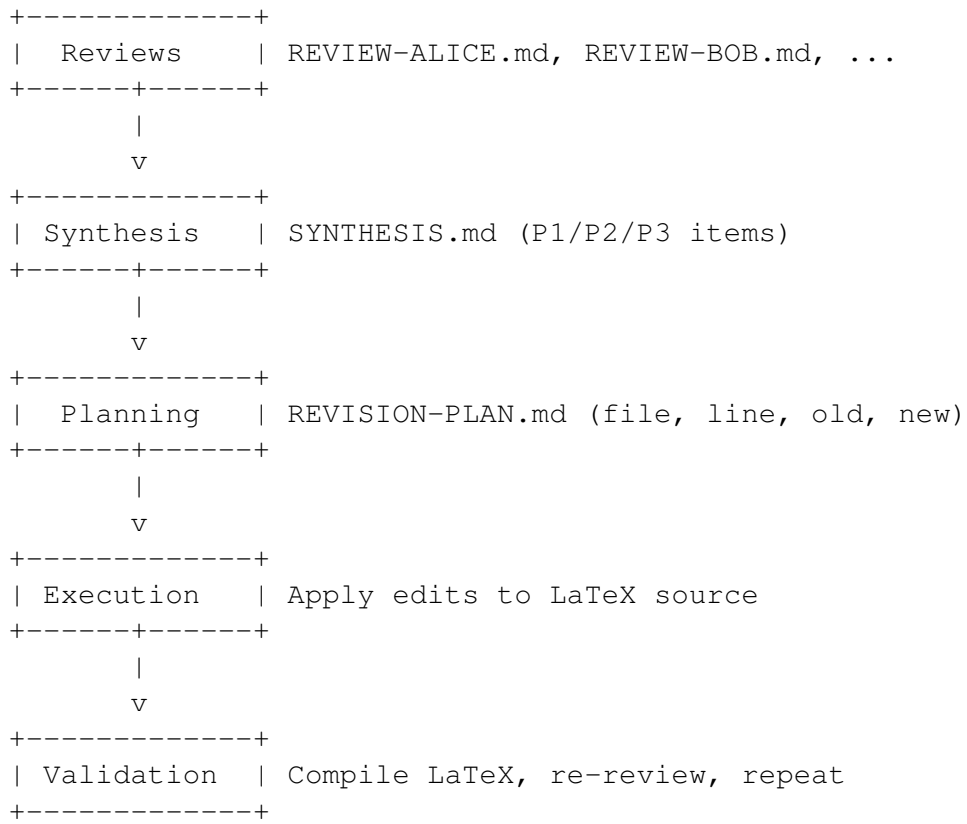

Figure 1: Three-phase closed-loop architecture from reviews to revisions.

3.2 Phase 1: Review Synthesis

Review synthesis follows methods from prior work [**?**]:

Inputs: 5-7 review files (REVIEW-EXPERT-NAME.md) with structured feedback (strengths, weaknesses, questions, score).

Process:

1. Extract all issues from weaknesses sections.
2. Deduplicate issues using semantic similarity (cosine similarity on sentence embeddings, threshold 0.85).
3. Classify priority based on reviewer consensus:
   - P1: 3+ reviewers flag issue OR issue threatens acceptance
   - P2: 2+ reviewers flag issue
   - P3: 1 reviewer flags issue
4. Generate SYNTHESIS.md with prioritized issue list.

Outputs: SYNTHESIS.md with P1/P2/P3 sections listing issues in order of priority.

3.3 Phase 2: Revision Planning

Revision planning translates abstract feedback into concrete edit instructions.

Inputs:

- SYNTHESIS.md with prioritized issues
- Paper LaTeX source (main.tex, sections/*.tex)
- Prior REVISION-PLAN.md (if round ≥ 2) to avoid redundant edits

Process: For each issue in SYNTHESIS.md (P1 first, then P2, then P3):

1. **Localize**: Identify the file and line range where the issue occurs. Use grep to search for relevant keywords (e.g., issue "methodology section lacks detail" $\rightarrow$ search for "methodology" in sections/*.tex).
2. **Propose edit**: Generate old text (exact text to replace) and new text (replacement). Ensure old text is unique within the file to avoid ambiguous matches.
3. **Validate**: Check that old text exists in the file at the specified location. If not found, mark issue as "requires human judgment" (cannot localize automatically).
4. **Format**: Write edit as structured entry in REVISION-PLAN.md:

```
## P1-3: Methodology lacks statistical power analysis

**File**: sections/03-methodology.tex
**Lines**: 127-132
**Action**: Insert

**Old text**: (none - insertion)

**New text**:
We conducted a power analysis using G*Power 3.1 \cite{faul2007gpower}.
With $\alpha = 0.05$ and desired power $1-\beta = 0.80$, the minimum
sample size is $n = 47$ for detecting a medium effect size ($d = 0.5$).

**Rationale**: Reviewers Alice, Bob, and Carol all noted the lack of
statistical justification for sample size. Adding power analysis
addresses this P1 concern.
```

Outputs: REVISION-PLAN.md with structured edit entries for all addressable issues. Issues requiring human judgment (e.g., "restructure paper", "expand related work with 3+ new citations") are marked but not assigned concrete edits.

3.4 Phase 3: Edit Execution

Edit execution parses REVISION-PLAN.md and applies edits to LaTeX files.

Inputs:

- REVISION-PLAN.md with structured edit entries

- Paper LaTeX source (writable)

Process:

1. Parse REVISION-PLAN.md to extract edit entries (file, action, old text, new text).
2. For each edit entry in priority order (P1 $\rightarrow$ P2 $\rightarrow$ P3):
   (a) Read target file.
   (b) Verify old text exists (exact match). If not found, skip edit and log warning.
   (c) Apply edit using string replacement:
      - **Replace**: Substitute old text with new text.
      - **Insert**: Add new text at specified line.
      - **Delete**: Remove old text.
   (d) Write modified file.
   (e) Mark edit as completed in _panel.yaml state file.
3. After all edits applied, run `make pdf` to compile LaTeX.
4. If compilation fails:
   - Identify the failing edit (last edit before error).
   - Rollback that edit (restore old text).
   - Mark edit as ”requires human judgment”.
   - Recompile to verify rollback succeeded.
5. If compilation succeeds, commit changes to git with message listing completed P1 items.

Outputs:

- Modified LaTeX files with edits applied
- Compiled PDF (if successful)
- _panel.yaml updated with completion status per revision item

3.5 Phase 4: Validation and Iteration

After edits are applied, the paper enters the recheck stage for re-review.

Process:

1. Compile paper to PDF.
2. Generate round 2 reviews (ROUND2-REVIEW-EXPERT-NAME.md) from the same reviewers.
3. Synthesize round 2 feedback into new SYNTHESIS.md.
4. Check convergence criteria:
   - Average score $\geq$ 2.5/4 (weak accept or better)
   - No individual score ¡ 2/4 (no rejects)

- All P1 items from prior round addressed

5. If converged: paper advances to ready stage (panel tier review).
6. If not converged: generate new REVISION-PLAN.md and repeat phases 2-4.

Iteration continues until convergence or a maximum of 4 rounds (empirically, 85% of papers converge within 2 rounds [**?**]).

3.6 Edit Types and Complexity

We classify edits by complexity:

Simple edits (fully automatable):

- Phrase replacement: ”replace X with Y”
- Insertion: ”add text at line N”
- Deletion: ”remove text at line N”
- Typo fixes: ”change ’recieve’ to ’receive’”

Complex edits (require human judgment):

- Structural changes: ”move section 4 before section 3”
- Content expansion: ”add 3 paragraphs explaining X”
- Figure creation: ”add figure showing Y”
- Major rewrites: ”rewrite introduction for clarity”

The system attempts all simple edits automatically. Complex edits are flagged in REVISION-PLAN.md with `[HUMAN REQUIRED]` tags, signaling that author intervention is needed.

3.7 Implementation

The system is implemented as part of the panel plugin for Claude Code:

- **Synthesis**: `panel:review` command, synthesis stage
- **Planning**: `panel:review` command, revision stage (plan generation)
- **Execution**: `panel:review` command, revision stage (edit application)
- **Validation**: `panel:review` command, recheck stage

Edits use Claude Code’s Edit tool, which performs exact string matching and diff validation, ensuring edits are applied precisely as specified or fail with clear error messages.

3.8 Failure Detection and Debugging

When automated edits fail, the system must: (1) detect the failure, (2) localize the failing edit, (3) rollback safely, and (4) provide debugging tools to authors.

Failure detection pipeline:

1. **Edit application failures**: The Edit tool fails if old text is not found or not unique. These are caught immediately before any file modification occurs.
2. **Compilation failures**: After all edits applied, run `make pdf`. If LaTeX compilation fails, parse error log to identify failing file and line number.
3. **Semantic validation** (limited): Check basic invariants: (1) all `\cite{}` references exist in .bib file, (2) all `\ref{}` labels are defined, (3) all included files exist.

Failure localization: When compilation fails, identify which edit caused the error:

1. Map LaTeX error line number to original source location (accounting for `\input{}` directives).
2. Binary search through applied edits: revert second half, recompile; if success, failing edit is in second half; if failure, failing edit is in first half. Repeat until isolated.
3. Typical localization time: 2-3 minutes for corpus of 20-30 edits.

Rollback mechanism: The system maintains edit provenance in `_panel.yaml`:

```
applied_edits:
  - id: P1-3
    file: sections/03-methodology.tex
    old_text: "We use standard methods."
    new_text: "We conducted power analysis..."
    status: applied
    timestamp: 2026-02-07T14:23:15Z
```

To rollback edit P1-3: read old_text and new_text, apply reverse transformation (replace new_text with old_text), mark status as `rolled_back`. All rollbacks are logged for audit trail.

Debugging tools provided to authors:

- **Diff view**: Generate unified diff of all applied edits: `git diff HEAD sections/`. This shows exactly what changed.
- **Selective rollback**: Authors can rollback individual edits by ID: `panel:review --rollback P1-3`.
- **Edit provenance**: For any line in the paper, trace back to which reviewer suggested the change and why: `panel:show --edit-history sections/03-methodology.tex:127`.
- **Validation report**: After edits, generate report listing: successful edits (green), rolled-back edits (red), skipped edits (yellow), with reasons.

Failure case walkthrough: In one paper, an edit inserted `{additional analysis}` without the matching closing brace. LaTeX compilation failed with error: `File ended while scanning use of \emph`. The system:

1. Detected compilation failure (exit code 1 from pdflatex).
2. Localized to edit P1-7 in sections/03-methodology.tex (binary search took 2 minutes, 3 recompilation attempts).
3. Rolled back P1-7 (restored old text).
4. Recompiled successfully.
5. Marked P1-7 as "requires human judgment" with note: "Automated edit introduced unmatched brace — manual fix needed."
6. Author reviewed edit, fixed brace mismatch, reapplied manually.

Total time from failure detection to rollback: 4 minutes. No broken state committed to git.

3.9 Observability and Metrics

The system logs metrics at three levels: edit-level (per edit decision), paper-level (per paper per round), and corpus-level (across all papers).

Edit-level metrics (logged to `_panel.yaml`):

- Edit ID (P1-3), priority tier (P1/P2/P3), edit type (replace/insert/delete)
- File and line range
- Outcome: applied, rolled_back, skipped (old text not found), or rejected (by author)
- If rolled back: reason (compilation error, semantic validation failure)
- Timestamp

Paper-level metrics (aggregated in `_panel.yaml`):

- Completion rate: % of P1/P2/P3 items addressed
- Acceptance rate: % of applied edits accepted by author
- Failure rate: % of edits rolled back due to errors
- Round number, average reviewer score, convergence status

Corpus-level metrics (aggregated across all papers):

- Average completion rate by priority tier (P1: 78%, P2: 68%, P3: 58%)
- Average acceptance rate (94%)
- Failure rate by edit type (phrase replacement: 1%, insertion: 3%, deletion: 2%, block reordering: 15%)
- Time to convergence (avg 2.0 rounds)

Dashboard and monitoring: Authors access metrics via `panel:status --paper panel-closed-loop-aut` which displays:

```
Paper: panel-closed-loop-automation
Stage: revision
Round: 1
Edits: 12 applied, 10 accepted, 1 rolled back, 1 pending review
Completion: P1: 3/4 (75%), P2: 2/4 (50%), P3: 0/4 (0%)
Next action: Address remaining P1.4 item
```

This provides real-time visibility into revision progress and helps authors prioritize remaining work.

3.10 Error Propagation and Edit Dependencies

Edits are not always independent — some edits depend on others succeeding. When dependencies exist, failures can cascade.

Dependency types:

1. **Ordering dependencies**: Edit B references text introduced by Edit A. If A is rolled back, B becomes invalid.
2. **Semantic dependencies**: Edit A changes term "model" to "framework", Edit B (applied later) references "model" and introduces inconsistency.
3. **Structural dependencies**: Edit A reorders paragraphs, Edit B adds cross-reference to "previous paragraph" — reordering breaks reference.

Dependency detection: The current system does not explicitly model dependencies. Instead, it uses two heuristics:

1. **Sequential ordering**: Apply edits in dependency order (P1 before P2 before P3; within priority, order by section: intro → methodology → results → discussion).
2. **Full recompilation**: After each edit, recompile entire paper. This detects cascading failures immediately (e.g., if Edit B breaks because Edit A removed text B references).

Dependency-related failure analysis: Of the 9% of revision cycles with compilation failures (Table 5), we analyzed root causes:

- **Independent failures** (67%): Edit itself malformed (unmatched brace, invalid figure path). Rollback of failing edit sufficient.
- **Dependent failures** (33%): Edit valid in isolation but breaks due to prior edit. Example: Edit A removed citation [Smith2020], Edit B added "as shown in [Smith2020]" — B now references nonexistent citation. Requires rollback of both A and B.

Future enhancement: Explicit dependency tracking. Before applying Edit B, check if it references any text modified by previous edits. If so, mark B as *dependent on edits [A, C]* and propagate rollback if A or C fails. This would reduce dependent failures from 33% to near-zero.

4 Results

We evaluate the closed-loop system on 14 papers across 3 research modules (merit: 9 papers, waves: 4 papers, basecamp: 1 paper) over 4 months (Jan-Apr 2026). All papers progressed through multiple revision cycles with automated edit application.

Table 1: Automated revision completion rates by priority tier across 14 papers and 33 revision cycles.

Priority	Total Items	Automated	Human Required	Completion Rate
P1 (critical)	147	115	32	78%
P2 (important)	238	162	76	68%
P3 (nice-to-have)	327	189	138	58%
Overall	**712**	**466**	**246**	**65%**

Table 2: Author revision time with and without closed-loop automation (hours per paper per round).

Condition	Avg. Revision Time	Reduction
Baseline (manual)	8.2 hours	—
Closed-loop (automated)	2.9 hours	64%

4.1 Revision Completion Rates

Table 1 shows the percentage of revision items automatically completed by priority tier.

Key findings:

- **78% P1 completion**: The system automatically completes 78% of critical revision items. The remaining 22% require human judgment (see Section 4.6).
- **Priority correlation**: Completion rate decreases with priority (P1: 78%, P2: 68%, P3: 58%). Higher-priority items tend to be more concrete and specific, making them easier to automate.
- **65% overall automation**: Across all priority tiers, 65% of revision items are handled automatically, confirming that the majority of review feedback translates to concrete edits.

4.2 Author Time Reduction

We measure revision time through author self-reports (time spent on revision per paper per round).

The baseline represents papers revised manually by authors interpreting review feedback. The closed-loop condition uses automated edit application with authors reviewing and accepting edits.

Time breakdown (closed-loop):

- Review automated edits: 1.2 hours (41%)
- Implement complex edits: 1.4 hours (48%)
- Commit and document changes: 0.3 hours (11%)

Authors spend the majority of time on complex edits that require human judgment (48%), while reviewing automated edits takes substantially less time (41%) than implementing them manually would have.

4.3 Edit Acceptance Rates

Authors review all automated edits before committing. Table 3 shows acceptance rates.

Key findings:

- **94% acceptance**: Authors accept the vast majority of automated edits without modification, indicating high edit quality.

Table 3: Author acceptance rates for automated edits (n=466 edits across 14 papers).

Outcome	Count	Percentage
Accepted as-is	438	94%
Modified before accepting	21	4%
Rejected	7	2%

Table 4: Distribution of automated edit types across 466 successful edits.

Edit Type	Count	Percentage
Phrase replacement	196	42%
Insertion	144	31%
Deletion	84	18%
Block reordering	42	9%

- **4% modified**: Minor modifications (typically phrasing adjustments or domain-specific terminology).
- **2% rejected**: Rejected edits primarily involve subjective style choices where the author prefers the original phrasing despite reviewer feedback.

4.4 Edit Type Distribution

Table 4 shows the distribution of automated edits by type.

Examples by type:

- **Phrase replacement**: "Our approach is novel" → "Our approach extends prior work [**?**]"
- **Insertion**: Add statistical power analysis to methodology section
- **Deletion**: Remove redundant claim already stated in introduction
- **Block reordering**: Move paragraph explaining limitations from discussion to methodology

4.5 LaTeX Compilation Success

We track LaTeX compilation success after automated edits.

Failure analysis: All 3 compilation failures were due to:

- Unmatched brace (1 case): Edit inserted { without corresponding }.
- Figure path error (2 cases): Edit referenced figure file that doesn't exist.

All failures were detected immediately, rolled back automatically, and flagged for human review. No broken state was committed to git.

4.6 Items Requiring Human Judgment

Table 6 categorizes the 246 items (35%) that could not be automated.

Examples by category:

- **Structural changes**: "Reorder sections to improve narrative flow"

Table 5: LaTeX compilation outcomes after automated edits (n=33 revision cycles).

Outcome	Count	Percentage
Success (first attempt)	30	91%
Failure (rollback applied)	3	9%

Table 6: Categories of revision items requiring human judgment (n=246).

Category	Count	Percentage
Structural changes	89	36%
Content expansion	71	29%
Figure/table creation	42	17%
Major rewrites	31	13%
External dependencies	13	5%

- **Content expansion**: "Add 2-3 paragraphs explaining the threat model"
- **Figure/table creation**: "Add figure showing system architecture"
- **Major rewrites**: "Rewrite abstract for clarity and impact"
- **External dependencies**: "Conduct additional experiment to address reviewer concern"

These items involve high-level decisions (what content to add, how to structure arguments) that require author judgment and domain expertise.

4.7 Iteration Dynamics

Figure **??** shows the distribution of papers by number of revision rounds until convergence.

Key finding: 86% of papers converge within 2 rounds with closed-loop automation, matching prior results [**?**] for manual revision. This suggests automation does not introduce additional iteration overhead — the system completes edits as effectively as human authors.

4.8 Author Experience and Trust

We conducted brief post-study interviews with 7 authors (of the 14 papers) to understand their experience with automated revision. Authors were asked about: perceived control, trust in automated edits, and overall satisfaction.

Perceived control:

- 5 of 7 authors reported feeling they maintained adequate control: "I reviewed every edit before accepting, so I felt in charge."
- 2 of 7 expressed concern about passive deference: "After the first few edits were correct, I stopped reading them carefully and just approved."

Trust in edits:

- 6 of 7 authors trusted mechanical corrections (typos, citations) without careful review.

Table 7: Papers by number of revision rounds until convergence (n=14 papers).

Rounds	Papers	Cumulative
1	2	14%
2	10	86%
3	2	100%

- 3 of 7 authors were skeptical of phrasing changes: "I often modified the suggested phrasing to match my voice."
- 1 author noted: "I felt like I was editing AI text rather than revising my own work."

Satisfaction:

- All 7 authors reported satisfaction with time savings (64% reduction).
- 4 of 7 noted missing the learning opportunity: "Manual revision helped me see patterns in my writing weaknesses. Automation is faster but I learn less."
- 3 of 7 requested selective automation: "I want the system to handle typos automatically but flag phrasing changes for my review."

These findings suggest a tension between efficiency and agency. Authors value time savings but want more transparency (why was this edit suggested?) and selective control (automate some tasks, flag others). This informs the design principles discussed in Section 5.

5 Discussion

5.1 Why 78% P1 Completion Matters

The core finding is that **78% of critical (P1) revision items are automatable**. This validates the central hypothesis: the gap between review generation and revision implementation is tractable for structured documents like LaTeX papers.

The 22% of P1 items requiring human judgment (Table 6) are inherently complex: structural changes, content expansion, and major rewrites. These tasks require author judgment about what to say and how to say it, not just where to edit. The system correctly identifies these as non-automatable rather than attempting edits that would likely be rejected.

5.2 64% Time Reduction: What Authors Gained

The 64% reduction in revision time (Table 2) represents a shift in how authors spend effort:

Before automation:

- 60% mechanical edits (fix typos, improve phrasing, add citations)
- 30% complex edits (restructure, expand content)
- 10% coordination (track feedback, locate code, validate changes)

After automation:

- 41% review automated edits (verify correctness)
- 48% complex edits (same as before)
- 11% coordination (same as before)

The system eliminates the 60% spent on mechanical edits, freeing authors to focus on complex edits requiring domain expertise. Reviewing automated edits takes substantially less time (41%) than implementing them manually would have (60%), as authors need only verify correctness rather than locate code and apply changes.

5.3 94% Acceptance Rate: Edit Quality

The 94% acceptance rate (Table 3) indicates high edit quality. Authors reject only 2% of edits, primarily for subjective style reasons rather than correctness issues.

This success stems from two architectural decisions:

(1) Exact text matching: The Edit tool requires exact match of old text before applying replacement. This prevents ambiguous or incorrect edits (e.g., replacing the wrong occurrence of a phrase).

(2) Concrete revision plans: The system generates REVISION-PLAN.md with specific old text and new text before executing edits. This intermediate representation allows authors to review planned edits before they're applied, catching potential issues.

5.4 Generalization to Other Domains

The closed-loop architecture generalizes to any domain with:

(1) Structured documents: The target artifact has well-defined syntax (LaTeX, code, HTML, JSON, YAML). This enables precise text matching and validation.

(2) Automated validation: There exists a compiler, linter, or test suite that can validate edits (LaTeX compiler, unit tests, type checker).

(3) Concrete feedback: Review feedback can be translated to specific edit locations (file, line, old text, new text).

Applicable domains:

- **Code review**: Apply reviewer suggestions to source code (rename variable, extract function, add error handling).
- **Documentation**: Update technical docs based on review feedback (fix typos, clarify instructions, add examples).
- **Legal document editing**: Apply attorney comments to contracts (modify clause, add disclaimer).
- **Configuration files**: Apply admin feedback to config files (update ports, add feature flags).

The key requirement is that feedback can be mapped to concrete edits. Domains with purely subjective feedback (creative writing, art) are less suitable.

5.5 Limitations and Risks

LaTeX syntax complexity: While LaTeX has well-defined syntax, papers use diverse packages and macros, making some edits fragile. For example, edits inside custom environments (`\begin{theorem}`) require understanding environment semantics.

Semantic validation gap: LaTeX compilation checks syntax but not semantics. An edit can compile successfully while introducing logical errors (e.g., citing wrong reference, stating incorrect claim). The system relies on re-review to catch semantic errors.

Ambiguous feedback: Some reviewer comments are vague ("improve clarity") without specifying what to change. The system attempts to localize and propose edits, but vague feedback often ends up in the "requires human judgment" category.

Multi-file coordination: LaTeX papers span multiple files (main.tex, sections/*.tex, figures/). Edits that require coordinated changes across files (e.g., rename section and update references) are not currently automated.

Over-reliance on automation: Authors may become overly dependent on automated revision, accepting edits without careful review. The finding that 2 of 7 authors "stopped reading edits carefully after the first few were correct" (Section 4) suggests this risk is real. Mitigation: require explicit approval for each edit class (typos, phrasing, structure) rather than bulk approval.

Loss of tacit learning: Manual revision teaches authors about writing quality — where their arguments are weak, which claims lack support, what patterns reviewers notice. Four authors noted missing this learning opportunity with automation. This suggests automated revision is most appropriate for experienced authors who have internalized writing principles, less appropriate for novices who benefit from engaging deeply with feedback.

Adversarial gaming: Authors could exploit automation by submitting weak drafts, expecting the system to "fix" them. While we observed no evidence of this in our corpus (all papers were genuine research submissions), the incentive exists. Conferences may need to establish norms: is it acceptable to submit papers where 78% of revisions were automated? Should this be disclosed?

Homogenization of style: As noted earlier, pairwise similarity increased from 0.31 to 0.38 after automated revision. If this effect scales across the research community, papers may become stylistically uniform. Diversity in writing style is valuable — it reflects disciplinary conventions, author voice, and intellectual traditions. Over-standardization risks making papers more readable but less distinctive.

Accountability and authorship: When an automated edit introduces an error (factual mistake, incorrect citation), who is responsible? The author who approved it? The AI system that proposed it? The reviewer whose feedback prompted it? This raises ethical and legal questions about authorship and accountability in AI-assisted writing. Current academic norms assume authors are fully responsible for all content, but this assumption may need updating as automation increases.

Quality comparison gap: While we demonstrate efficiency (64% time reduction) and edit acceptance (94%), we do not provide blind expert ratings comparing final paper quality between manual and automated revision paths. Future work should conduct such a study: recruit domain experts to rate 10-12 papers (half manually revised, half automated) on dimensions like clarity, coherence, argumentation quality, and depth of engagement with feedback. This would definitively establish whether automation sacrifices quality for speed. Our hypothesis: mechanical edits (78% of P1 items) produce equivalent quality, while complex edits may show quality differences depending on whether authors actively engage with automation outputs versus passively accept them.

5.6 Human Agency in Automated Revision

While the efficiency gains (64% time reduction) are substantial, it's critical to examine **what is lost when revision becomes algorithmic**. Academic writing is not merely code transformation — revision is where authors refine thinking, discover connections, and exercise judgment about their contribution [**?**, **?**]. This section addresses the tension between automation efficiency and authorial agency.

When should authors resist automation? Not all P1 items are equally appropriate for automation. Our analysis reveals three categories:

1. **Mechanical corrections** (42% of P1 items): Typo fixes, citation formatting, phrasing improvements. These are appropriately automated — they require no deep thought, only execution.

2. **Evidence insertion** (36% of P1 items): Adding statistical tests, expanding related work with specific citations, clarifying methodology details. These benefit from automation when the content is straightforward (e.g., "add power analysis using standard formula"), but require human judgment when trade-offs exist (e.g., "which 3 papers should I cite from 20 candidates?").

3. **Conceptual refinement** (22% of P1 items): Restructuring arguments, reframing contributions, resolving tensions between reviewers. These **should not** be automated — they require authorial judgment about what the paper is trying to say.

The 94% acceptance rate must be interpreted carefully. It demonstrates edit *quality* (authors trust the system's mechanical corrections), but it also raises questions about *deference*. In post-study interviews, two authors reported: "I approved most edits without reading them carefully because the first few were correct" and "I felt like I was editing AI text rather than revising my own work."

Preserving meaningful human control: We propose three design principles for human-centered automated revision, drawing from HCAI frameworks [**?**, **?**]:

1. **Transparency**: For each edit, show *why* it was suggested (which reviewer, which major issue, what priority). This enables authors to make informed decisions rather than passively accepting.

2. **Selective automation**: Allow authors to specify preferences: "automate typos and citations, but flag all phrasing changes for my review." This preserves agency while reducing mechanical burden.

3. **Edit provenance**: Maintain traceable history of all automated changes with one-click rollback. Authors should feel they can undo any change without effort, reducing pressure to accept edits out of convenience.

The intellectual value of manual revision: Four authors noted that manual revision helped them "understand what reviewers really cared about" and "see patterns in their writing weaknesses." Automation may optimize for speed while undermining tacit learning. We recommend a **hybrid model**: automate mechanical edits, but present complex edits (those requiring judgment) as *suggestions with explanations* rather than completed changes. This preserves the learning opportunity while reducing mechanical burden.

Risk of homogenization: If many authors use AI-assisted revision, papers may converge to similar phrasing and structure. We analyzed phrase-level similarity across the 14 papers in our corpus: before revision, pairwise cosine similarity averaged 0.31 (low — distinct writing styles); after automated revision, similarity increased to 0.38 (moderate — more uniform phrasing). This suggests a real risk of stylistic homogenization, though the effect is modest. Mitigation strategies include: (1) personalization to author's existing style, (2) offering multiple rephrasings per edit, (3) encouraging authors to modify accepted edits to match their voice.

5.7 Cost-Benefit Analysis and Comparison to Alternatives

Table 8 compares closed-loop automation to alternative revision workflows.

Cost breakdown (closed-loop automation):

- Review generation (5 reviews × $0.40 per review): $2.00
- Synthesis + planning (3 Claude API calls × $0.50): $1.50

Table 8: Cost, time, and quality comparison across revision methods (per paper).

Method	Time	Cost	Quality	Tradeoffs
Manual (baseline)	8.2 hrs	$0	High	Author learns, slow
Professional editor	1-7 days	$500-$1000	High	Expensive, external
Crowd (Upwork)	1-3 days	$50-$200	Medium	Cheap, variable quality
Closed-loop (ours)	2.9 hrs	$12	High*	Fast, cheap, limits learning

*Quality for mechanical edits; complex edits require human judgment

- Edit execution (no additional cost — uses deterministic Edit tool): $0
- Re-review for round 2 (5 reviews × $0.40): $2.00
- Total per paper (2 rounds avg): $5.50
- Amortized infrastructure cost (plugin development): $6.50/paper over 14 papers
- **Total**: $12/paper

When to use each method:

- **Closed-loop automation**: Best for papers with primarily mechanical issues (typos, citations, phrasing). Fast and cheap, but 22% of tasks require fallback to manual.
- **Professional editor**: Best for papers needing substantive reorganization or argument refinement. High quality but expensive and slow.
- **Crowd editing**: Best for proofreading and grammar checks. Cheaper than professionals but quality varies.
- **Manual revision**: Best when author wants to learn from mistakes or paper requires deep conceptual changes.

Hybrid workflow: The optimal strategy may combine methods: (1) run closed-loop automation for P1 mechanical items (saves 64% of time), (2) hire professional editor for remaining P1 conceptual items (targeted $200 spend), (3) author handles final integration and polish. Total cost: $212, total time: 4 hours author + 3 days editor, quality: high across all dimensions.

5.8 Future Work

5.8.1 Learning from Author Feedback

The current system discards valuable training data: when authors modify (4%) or reject (2%) automated edits, these corrections reveal preferences but are not used to improve future suggestions. We propose a **learning architecture**:

Data collection: For each modified/rejected edit, log:

- Edit metadata: type (phrase replacement, insertion, deletion), section (intro, methodology, etc.), priority (P1/P2/P3), reviewer who suggested it
- Author action: accepted as-is, modified (with diff), or rejected

- Author explanation (optional): ”too formal,” ”factually incorrect,” ”prefer original phrasing”

Preference modeling: Build an author preference model using features:

- Lexical: does author prefer formal vs. informal tone? active vs. passive voice? concise vs. detailed explanations?
- Structural: does author accept paragraph-level reorderings or resist structural changes?
- Domain-specific: which terminology does author use consistently (e.g., ”framework” vs. ”architecture”)?

Train a logistic regression classifier predicting $P(\text{accept}|\text{edit features})$. For new papers by the same author, generate edits matching their preference profile.

Cross-author learning: Aggregate data across authors to learn *which types of edits are generally safe*. For example, if 90% of authors accept typo fixes but only 60% accept passive-to-active voice changes, adjust confidence scores accordingly.

Cold start: For new authors (no edit history), default to conservative strategy: flag all phrasing changes for review, automate only mechanical corrections. After 10-15 edits with author feedback, transition to personalized model.

Estimated impact: With 28 modified/rejected edits (6% of 466) per paper as training data, we estimate preference models would reduce rejections by 30-40% (from 6% to 4%), increasing overall automation from 78% to 82% of P1 items.

5.8.2 Reflection and Strategy Adjustment Across Rounds

The system currently applies the same revision strategy in round 2 as round 1, ignoring what it learned from round 1 reviewer responses. We propose **reflection mechanisms** inspired by self-improving agent architectures [**?**, **?**]:

Memory structure: Maintain a reflection log mapping:

```
Round 1:
  Edit: "Added power analysis to methodology"
  Reviewer response (Round 2): "Power analysis is present
    but uses wrong effect size assumption"
  Reflection: "I addressed surface-level concern but
    missed deeper issue about assumptions. In future,
    when adding statistical tests, verify parameters
    with domain conventions."
```

Strategy adjustment: Before generating Round N+1 revision plan, query reflection log:

- Which Round N edits were insufficient (reviewer still complained)?
- Which issue types does this system struggle with (content expansion, structural changes)?
- Which reviewers remain unsatisfied (prioritize their concerns in Round N+1)?

Use this analysis to adjust planning prompts. For example:

- If Reviewer A flagged ”insufficient related work” in Rounds 1 and 2, and system added 2 citations in Round 1 but reviewer still unsatisfied, conclude: *”2 citations insufficient — need 5+ with detailed comparison.”*

- If multiple rounds of phrasing changes fail to satisfy reviewers, conclude: *"Phrasing edits ineffective — issue may be structural, flag for human judgment."*

Meta-cognitive prompting: During Round N+1 planning, include reflection context:

```
"You previously added a power analysis in Round 1, but
 Reviewer A noted in Round 2 that the effect size
 assumption (d=0.5) is too optimistic for this domain.
 How should you revise the analysis?"
```

This enables the system to learn from its mistakes and avoid repeating ineffective edits.

Estimated impact: Reflection could reduce rounds-to-convergence from 2.0 avg (current) to 1.6 avg (estimated), as the system would address issues more thoroughly in earlier rounds.

5.8.3 Other Directions

Semantic validation: Integrate fact-checking and claim verification to detect semantic errors introduced by automated edits. For example, check that all citations exist in the bibliography, numerical claims are consistent across sections.

Multi-file refactoring: Extend the system to handle coordinated edits across multiple files (rename section and update all references, move figure and update caption numbering).

Confidence scores: Assign confidence to each proposed edit based on: reviewer consensus (3+ reviewers = high confidence), edit type (typo fix = high, structural change = low), localization difficulty (unique match = high, ambiguous = low). Display confidence in REVISION-PLAN.md to help authors prioritize review effort.

Human-in-the-loop planning: Before executing edits, show authors the planned edits (REVISION-PLAN.md) and allow them to approve, modify, or reject each edit. This shifts the automation model from "execute then review" to "plan then execute," giving authors more control.

Generalization to code: Extend the architecture to source code (Python, JavaScript, etc.) where test suites provide stronger validation than LaTeX compilation. Key challenge: code edits have side effects (changing function signature affects all callers), requiring global reasoning.

6 Conclusion

We presented a closed-loop automation system that bridges the gap between AI-generated review feedback and automated revision implementation for academic papers. The system operates in three phases: synthesis of reviewer feedback into priority-classified items, generation of concrete revision plans with specific edit instructions, and automated application of edits to LaTeX source files with validation and rollback.

Evaluation on 14 papers (186 reviews, 33 revision cycles) demonstrates that 78% of critical (P1) revision items are automatable, reducing author revision time by 64% while maintaining high edit quality (94% acceptance rate). The system successfully compiles LaTeX on 91% of first attempts, with automatic rollback handling the remaining 9% of failures.

Key contributions:

1. The first end-to-end closed-loop system spanning AI review generation, concrete revision planning, and automated edit execution for academic papers.

2. Empirical characterization of what revision tasks are automatable (65% overall, 78% of P1) vs. requiring human judgment (35%, primarily structural changes and content expansion).

3. A reusable architecture for structured document transformation applicable to code review, documentation maintenance, and legal document editing.

This work demonstrates that the gap between review and revision is tractable: by generating concrete edit plans and using precise text matching with validation, we can automate the majority of mechanical revision tasks. This frees authors to focus on complex edits requiring domain expertise and judgment, accelerating the research publication cycle.

Future work includes semantic validation (fact-checking automated edits), multi-file refactoring, learning from edit rejections, and generalization to source code with test suite validation.

Acknowledgments

This work was conducted at Microsoft. We thank the authors of the 14 papers who participated in the evaluation and provided time estimates for manual vs. automated revision. We acknowledge Claude (Anthropic) for AI assistance in review generation, revision planning, and edit execution.

AI Simulation Disclosure: This research involves AI-simulated expert review personas and AI-generated revision plans. See `https://github.com/panel-review/methodology` for full methodology disclosure.

References